学习 • 执行 • 核对

AI Agent 实战入门

Python / RAG / MCP

24节图文课 · 3个基础练习

面向会一点 Python 的入门者

先运行基础练习，再阅读进阶案例

v1.0 / 2026.10.12 / 图文自学课程

目录

第01课 用三个小作品建立学习路线 3
第02课 把环境问题与业务问题分开 4
第03课 看懂一次模型请求的输入与输出 5
第04课 选择工作流，还是让模型选择行动 6
第05课 把函数写成模型能理解的工具 7
第06课 参数校验是执行前的最后一道关口 8
第07课 串起调用、观察和下一轮决策 9
第08课 让失败有分类，也有恢复办法 10
第09课 对话历史为什么不是自动记忆 11
第10课 用 SQLite 保存一轮完整对话 12
第11课 把有限上下文当成预算来管理 13
第12课 给长期记忆定义范围与失效规则 14
第13课 切分文档时保留事实和位置 15
第14课 分清关键词检索和向量检索 16
第15课 把资料放进回答流程，而不是放进承诺 17
第16课 核对引用，也核对结论 18
第17课 融合两路检索时先统一比较方式 19
第18课 重排解决的是候选顺序 20
第19课 检索纠错要有触发条件 21
第20课 按数据流诊断故障，而不是盲改提示词 22
第21课 理解 MCP 的主机、客户端与服务端 23
第22课 浏览器行动需要观察与结果核对 24
第23课 多服务协作先确定数据与权限边界 25
第24课 用分工与验收完成课程作品 26
继续学习与来源 27

第01课 用三个小作品建立学习路线

模块：认识 Agent | 建议用时：阅读25分钟 + 动手20分钟

学习目标

能把“学 Agent”拆成可以检查的结果；知道聊天、工具调用、记忆和文档问答各解决什么问题。

中文讲解

你会一点 Python，最容易遇到的困难通常不是不会写函数，而是不知道模型、工具和数据应该怎样连接。先给自己一个明确的终点：让程序完成一项任务，并留下能回查的过程。本课程用三个练习建立这条路径：计算工具展示执行，SQLite 展示状态保存，文档问答展示证据来源。

聊天模型接受消息并生成下一条消息。Agent 在此基础上增加行动：模型提出调用某个工具，程序验证参数，执行工具，再把结果交回模型。模型说“已经计算”不代表计算发生；你需要看到函数实际返回的结果。会话记忆解决重启之后数据还在不在，RAG 解决回答之前有没有找到外部资料。它们可以组合，但不是同一种能力。

学习时采用“先执行，再观察，再修改”的顺序。第一遍运行默认离线示例，排除网络、余额和模型波动。第二遍阅读关键函数，解释输入如何变成输出。第三遍只改变一个变量，例如把除数改成零，观察失败结果。能说明失败原因，比只展示一张成功截图更能证明你理解了程序。

流程：输入任务 → 程序状态 → 工具或资料 → 可检查的输出

每个作品都应有验收证据。计算助手记录 operation、a、b 和结果；记忆项目用两个独立进程验证恢复；文档问答显示文件名、行号和原文。课程中的三个练习是教学实现，不提供公网账号、浏览器自动化或生产环境保证。

示例与观察

在“03-练习项目”目录依次运行：

```
python tool_agent.py --operation multiply --a 18 --b 7
python memory_chat.py --session learner --message "我在学 RAG"
python document_qa.py "报销材料有哪些？"
```

第一个输出 126；第二个创建本地会话；第三个输出原文摘录及来源。默认运行均不请求模型。

动手任务

为三个作品各写一句验收条件，必须包含输入、预期行为和检查方法。然后写出你最想替换的真实学习任务。

参考答案

示例：输入 18×7，calculate 返回 126，并能在 observation 中查看；连续运行两次记忆程序，第二次输出包含第一次输入；问报销材料，sources 指向报销规则中的材料段落。真实任务可以是“从自己的产品说明里查退换条件”，先用虚构或公开资料测试。

常见问题

问：必须先学深度学习数学吗？答：本课程主要讲应用程序的数据流，先掌握函数、字典、JSON、终端命令即可。

问：离线运行也算模型 Agent 吗？答：离线模式验证程序结构，不具有模型自主决策能力。联网工具循环才让模型选择工具，两者的输出会明确标注。

第02课 把环境问题与业务问题分开

模块：认识 Agent | 建议用时：阅读25分钟 + 动手20分钟

学习目标

建立独立 Python 环境；辨认当前工作目录；理解环境变量和配置文件的区别。

中文讲解

一个脚本能运行，至少有三项条件：解释器存在、导入的模块存在、读取的数据路径正确。先从终端检查 Python 版本，不要在看到 ImportError 时就反复修改业务代码。三个练习只使用 Python 标准库，因此无需安装 Agent 框架、数据库客户端或向量库；SQLite 模块已经随常见 Python 安装提供。

解压后进入“03-练习项目”，建立独立虚拟环境。创建环境只是在本机放置一套解释器和包目录，不会替你配置云端模型。Windows 可直接使用虚拟环境的 python.exe，无需调整执行策略，也无需激活环境。路径带空格时用引号包住。

联网配置从当前终端的环境变量读取。目录中的 .env.example 只是变量名称示例，程序不会自动读取它；把真实密钥写进去也不会让脚本自动生效。变量作用于当前终端及其子进程，关闭终端后本次配置通常就结束。课程包不包含可使用的密钥。

流程：解压目录 → 解释器版本 → 独立环境 → 离线命令 → 可选联网配置

数据路径同样重要。记忆项目默认把数据库写到当前工作目录的 runtime；文档问答的默认示例目录则相对于脚本位置定位。学习排错时，记录执行命令、工作目录、Python 版本和错误类型四项信息，先定位是哪一层失败。

示例与观察

Windows PowerShell：

```
python --version
python -m venv .venv
.\.venv\Scripts\python.exe tool_agent.py
```

macOS/Linux使用python3与./.venv/bin/python，按相同步骤创建环境。 本包验收环境为 Windows / Python 3.12。其他系统命令供配置参考，未作同等实机验收。

动手任务

建立环境并运行计算示例。记录解释器路径，然后从另一个目录运行 document_qa.py，解释为什么默认示例资料仍能被找到。

参考答案

虚拟环境解释器在 .venv 下。文档目录使用 Path(__file__).resolve().parent 定位，取的是脚本所在目录，不是终端当前位置。memory_chat 的相对 --db 路径则取决于当前位置；需要固定位置时显式传入 --db。

常见问题

问：python 命令不存在怎么办？答：先安装官方 Python 3.12，或在 Windows 用 py -3.12；把命令中的 python 替换为实际解释器路径。

问：能复用现有项目的环境吗？答：可以，但独立环境更容易确认没有依赖被旧项目悄悄补齐，优先用独立环境验收。

第03课 看懂一次模型请求的输入与输出

模块：认识 Agent | 建议用时：阅读25分钟 + 动手20分钟

学习目标

辨认消息角色、文本结果和工具请求；知道模型名称与密钥是两项独立配置。

中文讲解

应用调用模型时，传递的不是一个完整的“聊天窗口”，而是一组消息。system 定义任务规则，user 提供当前问题，assistant 保存模型过去的答复，tool 保存程序执行结果。模型是否看见过去的内容，取决于本次请求有没有把那些消息发送出去，不能依赖网页聊天里形成的直觉。

配套 common.py 用标准库 urllib 发送 JSON 请求。密钥只放在请求头，模型名称从 DEEPSEEK_MODEL 读取。当前示例名称 deepseek-flash 取自制作时的官方文档；你的账户可用型号和接口参数可能变化，所以脚本要求显式配置型号，并在配置说明中保留官方文档链接。

返回内容需要先检查再使用。正常文本在 assistant 消息的 content 中，工具请求在 tool_calls 中。空结果、HTTP 错误、输出因长度上限中断，都不应当被保存为一条“完成”的答案。common.py 会把这些情况转成可读错误，并且不自动重试，以免一次命令触发你没意识到的额外调用。

流程：messages → JSON 请求 → HTTP 响应 → 检查完整性 → 文本或工具请求

联网练习会发送输入和必要的历史或资料片段到模型服务，并可能产生费用。先使用课程中的虚构资料。工具项目最多四次模型请求，记忆与问答每轮最多一次；这只是程序的调用次数上限，不是固定金额上限。

示例与观察

这段代码只构造消息，不联网：

```
messages = [
    {"role": "system", "content": "只回答 Python 学习问题。"},
    {"role": "user", "content": "字典怎么保存键值对？"},
]
assert messages[-1]["role"] == "user"
print(messages[-1]["content"])
```

联网配置步骤见“00-开始学习/配置与排错.md”，密钥不要放进截图或交付文件。

动手任务

阅读 common.py，找到请求头、模型字段、超时和 max_tokens。解释为何把 max_tokens 调小不等于降低输入长度。

参考答案

请求头含 Authorization；model 来自环境变量；超时为 60 秒；max_tokens 为 1200。输出预算限制生成长度，输入仍是 messages 的全部内容。完整响应检查会拒绝 finish_reason 为 length 的结果。

常见问题

问：设置了密钥还是报模型错误？答：确认 DEEPSEEK_MODEL 是当前账户可用名称，再检查官方说明。

问：HTTP 成功就代表答案正确吗？答：它只说明接口接受并返回了响应，事实正确、计算正确和证据支持要另行检查。

第04课 选择工作流，还是让模型选择行动

模块：认识 Agent | 建议用时：阅读25分钟 + 动手20分钟

学习目标

用任务稳定性判断控制方式；明确 Agent 的循环与停止条件。

中文讲解

工作流由程序预先决定步骤。例如“读取文档、切分、检索、展示摘录”，下一步始终固定。模型可以参与其中的某个节点，但不一定决定路径。Agent 的典型差别是：模型根据当前任务和工具结果，选择是否执行下一次行动。课程的文档问答基础版是检索工作流，联网计算助手包含由模型选择工具的循环。

先把任务中的稳定规则写出来。如果所有报销申请都必须包含三项材料，完整性检查适合普通 Python 逻辑。只有问题表达变化很大、需要动态选择工具时，再让模型参与决策。固定规则也更容易测试，不需要为“缺一个字段”付费调用模型。

Agent 循环必须有终点。模型输出文本可以视作一条结束候选，但还要检查内容是否完整。次数、超时和允许的工具集合应由程序控制；把“请别无限调用”写进提示词不能替代这些控制。配套工具助手只允许 calculate，并限制模型轮次和工具次数。

流程：固定工作流由程序选路；Agent 循环由模型提议、程序约束

不要用“越自主越高级”评价方案。一次任务只有一个确定计算时，直接调用函数更简洁。学习 Agent 的意义是理解动态路径如何被管理，能在必要的地方使用它，并知道在哪里退回确定性的代码。

示例与观察

固定流程示例：

```
def expense_check(form):
    required = {"invoice", "details", "approval"}
    missing = required - set(form)
    return {"ok": not missing, "missing": sorted(missing)}
print(expense_check({"invoice": "a", "details": "b"}))
```

这里无需模型，结果稳定为缺少 approval。

动手任务

分别给“固定格式报销检查”和“根据问题决定查资料或计算”选择控制方式，并写出一个停止条件。

参考答案

报销检查用固定规则；问题分流可用模型提出工具选择。后者至少设置最大轮次、允许工具列表和成功输出检查。若只有两种明确关键词，也可先写规则分流，再评估是否需要模型。

常见问题

问：用了 LangGraph 就一定是 Agent 吗？答：框架名称不能决定控制方式，要看路径由谁选择。

问：多步骤就是 Agent 吗？答：按固定顺序执行十个节点仍然可以是工作流，关键在决策权。

第05课 把函数写成模型能理解的工具

模块：工具调用 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

区分工具说明、参数结构和实际执行函数；理解注册表为何比函数名字更重要。

中文讲解

工具调用包含三个部分：说明告诉模型什么时候使用，参数结构描述输入，执行函数完成计算或访问外部系统。模型看到 calculate 的 JSON 描述，只能提出调用意图；它不会直接运行你的 Python 函数。程序必须把允许的工具名映射到明确函数，并检查输入。

配套工具接受 operation、a、b。operation 只允许四种基本运算，数字直接由程序计算。模型的责任是把“18 乘以 7”转换成结构化输入，程序的责任是得到 126。这个分工使结果可以独立验证，不必信任模型口算。

工具说明应写业务含义，而不是空泛地称其“强大”。比如“对两个数字做加减乘除”就能让模型判断是否适用。参数名要稳定；修改 a 为 first_number 时，schema、校验函数、测试和讲义示例要一起更新。

流程：工具说明 → 模型提出调用 → 注册表匹配 → 函数执行 → observation

不要根据模型输出动态导入模块或调用 eval。配套实现只分发给 calculate；未知工具返回结构化错误。上游案例可能使用其他实现方式，本课程独立练习采用明确的四种运算分支，让边界能直接读懂。

示例与观察

在练习目录运行：

```
from tool_agent import execute_tool
result = execute_tool(
    "calculate",
    '{"operation":"multiply","a":18,"b":7}'
)
assert result == {"ok": True, "result": 126}
print(result)
```

模型调用字段与这个函数的输入最终都要映射到同一份参数契约。

动手任务

为摄氏度转华氏度设计一个工具说明和参数结构，写出真正计算公式，不需要联网。

参考答案

工具名可为 celsius_to_fahrenheit，输入只含 celsius，公式为 celsius * 9 / 5 + 32。说明应强调输入单位；测试 0 得 32、100 得 212。校验应拒绝字符串“100℃”，由调用方先转换明确单位。

常见问题

问：函数有 docstring 就足够了吗？答：框架可能从 docstring 推导结构，但运行时仍应校验实际参数。

问：工具结果必须是自然语言吗？答：字典更容易检查成功、失败和具体数据，模型也能读取结构化结果。

第06课 参数校验是执行前的最后一道关口

模块：工具调用 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

拒绝缺失、多余、类型错误和越界参数；区分结构校验与业务规则。

中文讲解

JSON 能成功解析，只说明它符合 JSON 语法，不说明业务可以执行。“a 是字符串”“b 为零”“多了一个危险字段”都是语法合法但契约不合法的例子。因此 execute_tool 先解析 JSON，calculate 再检查字段集合、运算名、类型、范围和除数。

Python 的 bool 是 int 的子类，只用 isinstance(value, int) 会把 True 当作数字 1。本练习明确拒绝布尔值，也拒绝非有限值和过大的数字。数值边界并不适合所有业务，但教学项目需要一个可以解释、可以测试的范围。

校验失败应让调用方看懂如何修复。错误返回包含 ok=False 和 error，不能在错误时把 result 默认为零。因为零是合法计算结果，假零会让后续步骤误以为执行成功。

流程：JSON 解析 → 字段集合 → 类型 → 数值边界 → 业务规则

工具 schema 中的 required 和 additionalProperties 有助于引导模型，但不能替代 Python 侧校验。即使某个模型支持更严格的输出约束，你仍需要防止手工调用、版本变化和异常响应绕过说明。

示例与观察

执行四种失败输入：

```
from tool_agent import calculate, execute_tool
print(calculate({"operation":"divide","a":6,"b":0}))
print(calculate({"operation":"add","a":True,"b":2}))
print(calculate({"operation":"add","a":"3","b":2}))
print(execute_tool("calculate", "invalid json"))
```

每一种都应该返回错误，而不是把输入悄悄改成另一个计算。

动手任务

增加三个测试：缺字段、多字段、绝对值超过 10^12。说明为什么不能直接删除多余字段后继续执行。

参考答案

缺字段无法确定意图；多字段表示调用契约发生变化，应拒绝并让调用方修正；超界可能造成资源或数值问题。悄悄删除多余字段会隐藏版本不一致，也可能改变任务含义。

常见问题

问：为什么不自动把“3”转成 3？答：教学契约选择严格类型，转换责任清楚；若业务允许转换，应定义规则并单独测试。

问：所有错误都抛异常好吗？答：可预期的业务错误适合结构化返回；接口或配置无法继续时再用 CourseError 终止。

第07课 串起调用、观察和下一轮决策

模块：工具调用 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

阅读一条完整工具消息链；正确匹配 tool_call_id；控制模型轮次。

中文讲解

联网工具循环每轮发送当前消息列表。模型若直接返回文本，程序检查后结束；若返回 tool_calls，程序先保存这条 assistant 消息，再逐个执行工具，并把 tool 消息追加到历史。下一轮模型同时看到“我请求了什么”和“程序返回了什么”。

tool_call_id 是结果与请求之间的连接。一次模型响应可以提出多个调用，结果必须对应各自 ID；不能把所有结果合成一条没有 ID 的 user 消息。配套程序拒绝缺失和重复 ID，避免结果被错配。

“18×7，再加9”可以分两步完成：先得到 126，再把 126 与 9 相加得到 135。模型也可能先自行组织成其他合理调用路径，因此联网时不要要求固定调用次数；验收重点是执行结果和最终答案一致，并没有绕过工具。

流程：assistant.tool_calls → calculate → tool 消息 → 再次请求 → 最终文本

程序限制最多四次模型请求和八次工具调用。达到限制就停止并显示错误，不把最后一条工具结果冒充完整答案。上限是实验预算的一部分；生产系统还需要取消、耗时和重复调用管理。

示例与观察

离线计算第二步：

```
python tool_agent.py --operation multiply --a 18 --b 7
python tool_agent.py --operation add --a 126 --b 9
```

可选联网：

```
python tool_agent.py --online --question "18乘7之后再加9"
```

联网响应中 trace 展示工具结果，不打印密钥。

动手任务

画出两次计算所需的消息角色顺序。然后说明模型连续提出工具请求却没有最终文本时如何处理。

参考答案

角色顺序为 system、user、assistant(tool_calls)、tool、assistant(tool_calls)、tool、assistant(文本)。若始终不结束，轮次上限触发 CourseError。不能继续无限循环，也不能把一次中间结果标成最终完成。

常见问题

问：每次工具调用都需要模型吗？答：执行函数本身不需要；模型负责在循环中选择何时调用。

问：离线模式会自动理解中文运算问题吗？答：不会，它读取命令行指定的 operation、a、b，输出会注明模式。

第08课 让失败有分类，也有恢复办法

模块：工具调用 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

区分参数错误、工具错误和模型服务错误；避免无限重试和重复写入。

中文讲解

失败至少有三层。参数错误发生在执行之前，例如缺字段；工具业务错误发生在函数内部，例如除零；接口错误发生在模型请求，例如认证失败或超时。恢复方式不同，不能都归结为“再试一次”。

对于参数错误，模型可以读取 observation 并改正参数，但仍然受轮次上限约束。对于认证或余额问题，重复同一请求通常没有帮助，应先修改配置。对于超时，外部系统可能已经完成了操作，只是结果没有返回；如果工具会创建订单，直接重试可能重复执行。

教学计算是纯函数，相同输入总得到相同结果，没有外部写入，所以容易试验。把它换成发送邮件或提交表单时，要先设计请求标识、执行状态和确认边界。不要从“计算可以重试”推导“任何行动都可以重试”。

流程：失败分类 → 是否已执行 → 是否可重复 → 修正或停止

日志记录任务 ID、工具名、耗时和错误类别足以支撑初步排错。密钥不进入日志；涉及私有输入时也应限制原文记录。课程练习打印的是虚构任务和结构化结果，供你观察数据流。

示例与观察

构造未知工具和除零错误：

```
from tool_agent import execute_tool
print(execute_tool("send_email", "{}"))
print(execute_tool(
    "calculate", '{"operation":"divide","a":9,"b":0}'
))
```

前者未执行任何动作，后者识别到合法工具但业务条件不满足。

动手任务

为“工具参数错误”“HTTP 401”“HTTP 超时”各写一个恢复动作。再写一个不能盲目重试的工具例子。

参考答案

参数错误修正字段后重提；401 检查密钥；超时检查任务是否执行及外部状态后再决定。创建订单、支付或发信都可能产生重复副作用，需要操作标识与状态查询。

常见问题

问：自动重试总能提高成功率吗？答：不一定，配置错误不会靠重试解决，写入工具还可能重复动作。

问：课程计算函数为什么不做复杂重试？答：它在本机立即执行，主要失败可直接分类，不需要引入额外调度机制。

第09课 对话历史为什么不是自动记忆

模块：会话与记忆 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

区分进程内列表、持久化记录和本次模型上下文；解释重启与记忆的关系。

中文讲解

messages 列表只存在于当前 Python 进程里，退出后通常消失。把它存进数据库，可以在下一次运行恢复记录；但模型不会自动读取数据库。应用仍需要查询记录、选择要发送的内容，再构造 messages。这三个位置不能混为一谈。

课程的离线记忆演示只回放上一次用户输入，验证数据库是否跨进程保留。它没有语义理解，也没有从历史里提炼长期偏好。联网模式会把近期完整对话传给模型，让你观察“发送的上下文”与“数据库里全部历史”之间的差别。

一条对话最好保存角色和文本，而不是只保存一个拼接长字符串。角色明确以后，恢复时才能区分用户请求与助手答复，也容易检查重复或缺失。会话 ID 决定查询哪组记录；本地练习的 session 只是分组键，不是登录身份或权限系统。

流程：本次输入 → 读取会话记录 → 选取上下文 → 得到答复 → 保存记录

完成本课后，你应该能指出一次“模型忘记”的原因属于哪个层次：没有保存、没有恢复、没有发送，还是模型收到后没有正确使用。先检查前面三项，再评价模型能力。

示例与观察

用两个独立命令验证：

```
python memory_chat.py --session alice --message "我想做文档问答"
python memory_chat.py --session alice --message "我上次说了什么？"
python memory_chat.py --session bob --message "开始学习"
```

alice 的第二次运行回放前一句；bob 不应该读到 alice 的记录。

动手任务

画三个盒子：进程列表、SQLite、模型输入。分别写出重启后保留什么，以及哪一步决定模型能看见什么。

参考答案

进程列表退出后消失；SQLite 文件保留历史；模型输入由程序每次重建。数据库保存了事实，不意味着该事实必然出现在本轮请求，也不意味着模型永远记得它。

常见问题

问：session 不同就等于账号隔离吗？答：本项目只有分组过滤，没有认证。持有数据库文件的人可以读取所有内容。

问：离线回放能证明长期记忆了吗？答：它证明保存和恢复，不能证明语义检索、偏好学习或记忆准确性。

第10课 用 SQLite 保存一轮完整对话

模块：会话与记忆 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

理解参数化 SQL、稳定排序和事务；验证失败请求不会留下半轮记录。

中文讲解

SQLite 把数据保存在本地文件，适合单人学习和小规模原型。配套 messages 表包含递增 id、session、role、content。id 用于确定消息顺序，session 用于过滤，role 用于恢复消息角色。不要依赖数据库返回的自然顺序，查询里显式写 ORDER BY id。

SQL 使用问号参数占位，再把值作为元组传入。这样输入中的引号会作为数据处理，不会改变 SQL 结构。直接把用户内容拼进 SQL 字符串容易造成语法问题和注入风险。学习时即使只是虚构输入，也应养成参数化习惯。

一轮对话在获得完整答复后一起保存两行：user 与 assistant。数据库事务使这两行作为一个整体提交。若联网失败，程序不保存这轮输入，避免未来恢复出一个没有回答的半轮记录。课程选择这个简洁规则，生产系统可另设明确的 pending 状态。

流程：生成成功 → 事务开始 → 写入用户与助手 → 提交 → 按 id 恢复

查看文件大小不是验证记录完整性的好办法。用 --history 查询角色、文本和顺序，再用独立进程运行确认持久化。数据库文件属于你的运行数据，不要和空白课程源码一起发给别人。

示例与观察

查看当前会话：

```
python memory_chat.py --session alice --history
```

参数化查询的核心形式：

```
rows = conn.execute(
    "SELECT role, content FROM messages WHERE session = ? ORDER BY id",
    (session,)
).fetchall()
```

该片段里的 conn 与 session 来自 memory_chat.py 的 history 函数。

动手任务

输入一句包含单引号的文本，确认能正常保存。随后切换 session，验证记录没有串组。

参考答案

例如输入“我在读 Python's tutorial”。SQL 参数化使引号作为普通文本保存。新的 session 查询为空或只包含该组自己的对话，不能出现旧会话内容。

常见问题

问：两个进程同时写会怎样？答：SQLite 会管理写锁，但本练习没有多用户服务和复杂重试，验收范围是单人串行运行。

问：删除终端记录会删除数据库吗？答：不会，文件仍在 --db 指定的位置。

第11课 把有限上下文当成预算来管理

模块：会话与记忆 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

选择近期完整轮次；理解字符预算与 token 预算的区别；说明被截掉的信息。

中文讲解

把所有历史每次都发给模型，会让请求越来越长，费用和延迟也可能增加。模型的输入容量有限，即使接口接受长上下文，早期信息也未必被稳定使用。先明确哪些内容值得保留，比盲目增加窗口更有用。

配套 recent_context 从最近的 user/assistant 配对向前选，最多六条消息，并设置 6000 字符的近似预算。它不会把一轮对话从中间截断，也不会把单个 assistant 消息悬空放在最前面。当前用户问题另外追加，数据库中的旧历史并未删除。

字符不是 token：中文、英文、数字和符号的分词不同。这里用字符预算是为了把控制机制讲清楚，不承诺它对应某个精确接口预算。需要严格费用管理时，应使用当前模型的计量办法并读取接口 usage。

流程：全部历史 → 最近完整轮次 → 预算裁剪 → 当前问题 → 模型输入

摘要是另一种压缩方式，但会引入新错误。摘要里漏掉“只用公开资料”这样的约束，后续模型就无法恢复。保存摘要时应保留原始历史和摘要版本，区分事实、用户要求和助手建议。课程基础项目暂不自动摘要，避免把生成误差藏进记忆。

示例与观察

检查裁剪结果：

```
from memory_chat import recent_context
history = []
for i in range(5):
    history += [{"role":"user","content":f"问题{i}"},
                {"role":"assistant","content":f"答复{i}"}]
selected = recent_context(history, limit=4)
assert len(selected) == 4
assert selected[0]["content"] == "问题3"
```

只保留最后两轮，旧记录仍可从数据库查询。

动手任务

设计一个五轮对话，把重要约束放在第一轮。裁剪到最近两轮后，解释模型为什么可能不知道约束。

参考答案

约束被裁剪后没有出现在本次 messages，模型没有可靠来源。可将经过确认的任务约束放入独立状态，每轮显式发送；不要要求模型凭空记住未传入的历史。

常见问题

问：最多六条消息是六轮吗？答：不是，user 与 assistant 各算一条，通常是三轮。

问：裁剪会删除数据库里的历史吗？答：不会，它只影响本轮模型输入。

第12课 给长期记忆定义范围与失效规则

模块：会话与记忆 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

区分历史记录、任务状态与长期事实；识别过期、冲突和跨会话泄漏。

中文讲解

长期记忆不是把全部聊天永久堆在一起。历史记录保存发生过的对话，任务状态保存正在做什么，长期事实保存可能在未来重复使用的信息，例如用户确认的学习目标。三者在来源、保存期限和更新方式上都不同。

一条长期事实至少应知道内容、来源、确认时间和适用范围。“预算是1000元”可能只适用于某次旅行；如果直接变成永久偏好，下次任务会沿用错误约束。新旧信息冲突时，需要明确哪个更近、哪个被用户确认，而不是把两条都拼进提示词。

检索到记忆也不代表必须使用。先检查任务是否相关、信息是否仍有效、会话或用户边界是否允许。课程的 session 过滤展示了最基本的分组思想，但不包含加密、认证或权限判断。

流程：候选事实 → 明确来源 → 确认范围 → 保存 → 到期或更新

可从小表格开始设计，不必立刻安装复杂记忆框架。对学习目标设置“用户主动更改时更新”，对当次预算设置“任务结束失效”。这种明确规则让程序更容易解释，也能避免无关的旧事实干扰新任务。

示例与观察

教学事实对象：

```
fact = {
    "content": "本次旅行预算1000元",
    "source": "用户本次输入",
    "scope": "trip-001",
    "status": "confirmed",
}
print(fact["scope"])
```

这是设计示例，配套 SQLite 项目实际只保存对话消息。

动手任务

给“学习方向”“本次旅行预算”“模型猜测的用户职业”各选择保存策略。

参考答案

学习方向经用户确认后可长期保存并允许修改；预算限制在本次任务；猜测的职业不应当变成确认事实。保存时附来源并为用户保留修正方式。

常见问题

问：RAG 和长期记忆一样吗？答：二者都可能涉及检索，但 RAG 常查外部知识，长期记忆还涉及个人事实的更新、范围和生命周期。

问：本包有自动学习用户偏好吗？答：没有，基础项目验证历史保存、恢复和上下文选择。

第13课 切分文档时保留事实和位置

模块：文档问答 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

从文件构建片段；保留文件名与行号；辨认切分过小和过大的后果。

中文讲解

文档通常比一次回答所需的资料长。切分的目标是得到能独立理解、便于检索的小片段。太小会把规则与例外拆开，太大则让无关内容占据上下文。先按自然段和标题观察文档结构，再决定预算，而不是照搬某个固定字符数。

课程示例只读取 UTF-8 的 .md 和 .txt，遇到空行或达到约 650 字符时结束当前片段，保存相对文件路径和原始行号。它不是 PDF 解析器，表格和长行也不做智能重排。若一行本身很长，先在原文里按段落拆开更容易理解和检查。

来源位置要在切分时保存。生成以后再猜片段来自哪里，容易出现引用失真。重复的句子也可能出现在多个章节，不能只用文本搜索来确定唯一来源。

流程：UTF-8 原文 → 段落边界 → 片段文本 → 文件与行范围

更新文档后，旧行号可能移动。课程基础版每次运行重新读取文件，因此不维护陈旧索引；生产索引需要记录文件版本或内容哈希，并在修改后重建相应记录。

示例与观察

查看生成的片段：

```
from document_qa import chunks_from_directory
chunks = chunks_from_directory("sample_docs")
for chunk in chunks:
    print(chunk.file, chunk.start, chunk.end)
    print(chunk.text)
```

打开对应原文件，检查行范围是否确实包含显示的内容。

动手任务

把“私人聚餐不报销”写成紧挨费用规则的例外，比较分在同一段和分在相隔很远的段落时的检索结果。

参考答案

例外与规则在同一段时更容易一起检索。远距离切分可能只召回规则而遗漏例外；可以改善文档结构、扩大相邻片段或用结构化条件，不能只依赖模型补出缺失内容。

常见问题

问：650 字符是最佳切分大小吗？答：它是教学默认值，最佳大小要用自己的文档和问题验证。

问：扫描 PDF 能直接放进目录吗？答：本项目不会读取 PDF，需先解析或 OCR 并检查结果。

第14课 分清关键词检索和向量检索

模块：文档问答 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

解释两种检索的匹配依据；用同义问题观察关键词基线的局限。

中文讲解

关键词检索比较问题与文档中的词。课程基线提取英文单词和中文连续二字片段，再用重叠数量排序。它容易解释：你能指出哪些字词贡献了匹配；但不理解完整语义，“报销凭证”与“费用材料”未必能稳定匹配。

向量检索先把文本转换成一组数字，再比较问题向量与文档向量。它可能召回表达不同但语义相近的片段，也可能召回词义相近但业务无关的资料。向量模型不是事实判断器，相似度高不表示文档回答了问题。

同一个索引的文档向量与查询向量必须使用兼容的嵌入模型和处理方式。更换模型后，通常需要重新生成索引，不能只改查询侧配置。维度相同也不证明两个模型的向量空间相同。

流程：关键词比较词项；向量比较嵌入表示；两者都要查相关性

先保留关键词基线，准备包含精确术语、同义表达、无答案的问题，再比较改进。课程不内置向量服务，向量与混合检索的实现参考案例手册，不把当前代码称为向量 RAG。

示例与观察

在配套项目里对比：

```
python document_qa.py "报销材料有哪些？"
python document_qa.py "费用申请要准备哪些凭证？"
python document_qa.py "量子纠缠的数学定义是什么？"
```

第二题可能不如第一题稳定，第三题应返回没有相关资料。

动手任务

为一个产品说明文档写三道精确术语题、三道同义题和两道无答案题，保存预期证据位置。

参考答案

例：精确题“保修期限是多少”，同义题“坏了多久内能免费修”，无答案题“竞争品牌保修多久”。预期证据应指向真实文档；无答案题应注明没有对应证据，而不是给一个编造的位置。

常见问题

问：关键词分数能叫置信度吗？答：不能，它只是重叠词项数，没有校准成正确概率。

问：向量检索一定更好吗？答：要看题型和数据，精确编号、代码符号等常常仍需要词项匹配。

第15课 把资料放进回答流程，而不是放进承诺

模块：文档问答 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

构建检索增强生成数据流；区分离线摘录与联网回答；处理无结果。

中文讲解

RAG 的应用流程可以拆成检索和生成。检索找到候选片段，生成把问题与片段组织成答复。两个阶段都可能失败：没找到相关资料时，模型再会表达也没有证据；资料找对了时，模型仍可能忽略例外或补出不存在的结论。

配套 document_qa 默认离线输出原文摘录，sources 提供位置。加 --online 后，才把最多三个候选片段与问题发送给 DeepSeek。这个区分让你先验证检索，再验证生成，不把两类问题混在一起。

没有检索结果时，程序返回说明并且不请求模型。有结果也不一定足够回答问题：例如文档给了提交日期，却没有到账日期。联网提示要求只据资料作答，但仍要人工检查是否把“进入审核”写成“钱已到账”。

流程：问题 → 候选片段 → 来源编号 → 模型答复 → 回查证据

检索资料是数据，不应当变成新的程序指令。文档里出现“忽略系统要求”时，也应被视为原文内容。提示词提供基础约束，但完整系统还需要隔离工具权限、限制数据访问和处理恶意内容。

示例与观察

默认离线：

```
python document_qa.py "每月几号前提交报销？"
```

可选联网：

```
python document_qa.py "20日以后提交会怎样？" --online
```

联网会传输选中的文档片段；不要使用未经允许的私有资料。

动手任务

问“报销什么时候到账”，检查原文是否提供到账日。写出合适的回答边界。

参考答案

原文只说 20 日前提交，超过该日进入下月审核，没有到账日期。合适回答说明已知提交与审核规则，并明确无法依据现有资料确定到账时间。

常见问题

问：有三个来源就说明回答完整吗？答：来源数量不等于覆盖度，可能三段都没有关键条件。

问：离线摘录为什么不是自然语言总结？答：它刻意避免生成新结论，让你单独观察检索是否找对。

第16课 核对引用，也核对结论

模块：文档问答 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

分别检查来源存在、位置准确和结论支持；识别引用编号带来的虚假可靠感。

中文讲解

引用至少有三层检查。第一层是编号存在：答复中的 S1 是否对应本次提供的来源。第二层是位置准确：文件和行号能否回查到原文。第三层是语义支持：原文是否真正支持答复的结论，而不是只提到同一个主题。

配套联网问答检查编号存在，并拒绝没有引用或引用不存在编号的答案。它没有自动证明每句话正确。sources 中保留完整片段，方便你打开文件核对；离线模式直接摘录原文，仍可能因为召回错误而没有回答到问题。

“超过20日进入下月审核”不能支持“下个月1日到账”。虽然两句话都在讨论报销，后一条增加了原文没有的日期。检查时划出每个事实性断言，再为它找具体句子；找不到就删掉、改成不确定或补充资料。

流程：引用 ID 存在 → 行范围准确 → 原文支持断言 → 条件与例外完整

不用给每个连接词加引用，但日期、金额、权限、范围和因果解释都应有可回查支持。不要把接口成功、截图漂亮或引用数量当作内容质量的证明。

示例与观察

用四列记录核查：

答复断言	来源	原文支持	处理
需电子发票	S1	有明确材料要求	保留
下月1日到账	未找到	资料没有到账日	删除

把 sources 中的行号与 sample_docs 原文件对应起来。

动手任务

给“私人聚餐可报销，只要有发票”这句话找支持，并指出错误部分。

参考答案

教学原文明确私人聚餐不在报销范围，发票不能改变费用范围。这句话应判为与原文冲突，不能因为引用了报销材料段落就接受。

常见问题

问：citation_check 显示通过，是答案正确吗？答：它只校验来源 ID，程序会明确提示仍需人工回查。

问：能把源文件名放在答案后面代替编号吗？答：可以设计其他格式，但必须保持来源与具体断言的对应，不能只列一堆文件。

第17课 融合两路检索时先统一比较方式

模块：RAG 优化 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

理解混合召回、去重和排名融合；避免直接相加不可比较的分数。

中文讲解

关键词擅长精确词项，向量擅长相近语义，混合检索希望覆盖两类问题。通常先各取候选，再按片段 ID 去重，最后融合或重排。不是把两个完整答案拼在一起，而是先融合证据候选。

两路分数可能处于不同尺度。一个是词项相关分，一个是余弦相似度，直接相加会让数值较大的那一路主导。可先使用基于名次的融合，例如每个候选从各路获得 1/(k+rank)，再累加。rank 从 1 开始，k 是平滑常数。

融合后仍要控制重复信息。同一段文档在两路都出现时应作为一个来源，而不是把它算成“两份独立证据”。相邻重复片段也会挤占上下文，应按事实覆盖检查。

流程：词项召回 + 向量召回 → ID 去重 → 融合排名 → 候选证据

用固定题集比较词项、向量与混合方法，尤其看精确编号与同义表达的差别。记录原始排名和最终排名，才能解释某条证据为何上升或下降。课程基础代码不包含这个融合算法，下面是独立的教学小实验。

示例与观察

基于名次融合：

```
def fuse(rankings, k=60):
    scores = {}
    for ranking in rankings:
        for rank, item in enumerate(ranking, 1):
            scores[item] = scores.get(item, 0) + 1 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)
print(fuse([["A", "B"], ["B", "C"]]))
```

B 在两路均被召回，一般排到更靠前。

动手任务

为编号查询与同义查询各造一份两路排名，运行 fuse，解释结果。

参考答案

编号题的关键词第一名可能最可靠；同义题向量第一名可能更有帮助。融合只是排序方法，仍须检查证据是否回答问题。两路同时排前不代表来源独立或结论正确。

常见问题

问：混合一定比单路慢很多吗？答：要测实际检索与服务耗时，不能仅凭算法名称判断。

问：平滑常数60是通用最优值吗？答：它是常见实验起点，需用自己的题集验证。

第18课 重排解决的是候选顺序

模块：RAG 优化 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

区分召回和重排；说明重排无法找回缺失候选；把耗时纳入比较。

中文讲解

召回先从大量文档中找到一个较小候选集，重排再更精细地判断候选与问题的相关性。分工的原因是：精细模型逐对比较所有文档通常成本较高，先缩小范围更实用。

如果关键文档没有进入候选，重排无法凭空找回。比如问题问“例外退款条件”，召回的十个片段都只有一般退款规则，重排只能在不完整的资料里排序。此时应检查查询、切分和召回数量，而不是继续增加重排提示。

候选数量与最终上下文数量不是一回事。可以先召回十条，再取重排后的三条发送给生成模型。最终选择还要考虑重复、来源多样性和事实覆盖，不能只按分数截取三段同义描述。

流程：粗召回 → 候选相关性比较 → 重排 → 覆盖检查 → 生成上下文

案例手册中的 Hybrid RAG 使用单独重排组件；上游模型与接口名称属于固定快照。接入你自己的服务时，需要另外验证兼容性、费用与延迟。本课先通过一个人工重排练习理解目标。

示例与观察

人工候选集：

候选	内容	对“超过20日怎么办”的相关性
A	需提交费用明细	低
B	超过20日进入下月审核	高
C	办公用品可报销	低

选择 B 后仍须说明资料没有给出到账日期。

动手任务

从自己的问题中取五个候选，先人工排序，再写出每条排序理由。

参考答案

理由应对应问题要求，例如“包含时间条件与后续处理”，不要只写“分数高”。如果五条都不含答案，结论应是召回失败，不能硬选一条当正确证据。

常见问题

问：重排分数就是答案置信度吗？答：不是，它评价问题与片段的相关性，生成正确性是另一项检查。

问：重排越多候选越好吗？答：可能增加覆盖，也会增加成本与噪声，用题集衡量收益。

第19课 检索纠错要有触发条件

模块：RAG 优化 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

给资料相关性判断设计分支；限制查询改写与外部搜索；保持原始任务。

中文讲解

检索纠错的出发点是：第一轮资料可能不够相关，需要判断后决定是否重新找。可以把状态写成 question、documents、relevance、rewrite_count。先保留原始问题，再保存改写后的查询，方便检查改写有没有偏离任务。

改写不等于把问题改成更容易回答的另一道题。“20日后提交什么时候到账”若被改写成“报销提交截止日期”，虽然能找到资料，却丢失了用户关心的到账问题。合格改写应保留对象、条件和目标，只调整检索表达。

外部搜索也要有边界。如果本任务只允许依据公司内部规则，搜索公开网页不会补出公司的真实政策。可以选择返回信息不足；只有任务允许外部资料时，才搜并保留来源。搜索结果中的内容也不是新的系统指令。

流程：检索 → 判断相关性 → 有证据则回答；不足则改写一次或说明缺失

上游 Corrective RAG 案例有检索、打分、改写、搜索和生成节点，并不是无限自我修复循环。课程建议先设一次改写上限，记录每次分支的触发理由，再评估是否需要复杂循环。

示例与观察

一次改写的状态示例：

```
state = {"original":"报销凭证有哪些？",
        "query":"报销材料 电子发票 费用明细",
        "rewrite_count":1}
assert state["rewrite_count"] <= 1
print(state["original"])
```

这是流程设计示例，不包含联网搜索。

动手任务

给“报销材料”与“到账日期”各设计一个相关性检查条件，以及资料不足时的动作。

参考答案

材料题需要召回列明材料的段落；到账题需要明确到账日期或规则。若只有审核信息，到账题应说明缺失。允许公开检索也不能拿别家公司政策当成本公司的答案。

常见问题

问：模型能给资料打分就能保证纠错成功吗？答：不能，打分本身也可能错，需要人工标注题集验证。

问：搜索失败能用常识补答案吗？答：证据型任务应说明资料不足，不能偷偷更换回答依据。

第20课 按数据流诊断故障，而不是盲改提示词

模块：RAG 优化 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

把失败定位到读取、切分、召回、选择或生成；建立小题集与回归记录。

中文讲解

面对错误回答，先保存一次完整样本：问题、原始资料版本、候选片段、排序、最终输入、回答和错误类型。没有这些数据，只凭最后一句“答错了”很难知道该改哪里。

按阶段提问：文档是否读对；关键事实是否在同一片段；正确片段是否被召回；是否在重排后被挤掉；生成是否忽略条件。先检查最早失败的位置。缺文档的问题不能靠提示词解决，引用错误的问题也不能只靠增加召回数量。

建立至少八题的小题集，含精确表达、同义表达、例外条件和无答案题。每题写出预期证据和可接受答案边界。调一个参数后重跑同一题集，记录改善了哪题、退化了哪题，不要只展示你挑中的成功题。

流程：失败样本 → 分阶段核查 → 最早失败点 → 单项修改 → 同题回归

检索指标与答案指标要分开。正确证据进了 top-k，可以算召回成功；回答是否使用正确证据、有没有补造事实，还需要内容检查。测试通过只能支持它实际覆盖的行为。

示例与观察

可复制的故障记录模板：

项目	记录
问题	报销多久到账
预期证据	原文没有到账日
实际候选	提交规则
错误	把审核日期当到账日期
修改	明确信息缺失，增加无答案题

不要把课程虚构规则应用到真实公司报销。

动手任务

建立八题回归表，并挑一题故意修改文档，验证记录能发现行为变化。

参考答案

可用材料、截止时间、超期、费用范围、私人聚餐、PDF支持、同义材料、到账日期八题。改动截止时间后应更新预期；若结果仍指向旧内容，检查缓存或索引版本。

常见问题

问：测试越多越好？答：优先覆盖重要失败模式，重复同一实现逻辑的测试不一定增加可信度。

问：RAG 故障诊断案例自动修复系统吗？答：上游主要生成故障分析建议，仍需工程师核对与实施。

第21课 理解 MCP 的主机、客户端与服务端

模块：MCP 与协作 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

辨认 MCP 的三个参与者和三类能力；区分协议、工具实现和模型决策。

中文讲解

MCP 提供应用连接外部能力的共同约定。Host 是承载 AI 应用的主程序，Client 管理与某个服务端的连接，Server 提供工具、资源或提示模板。模型仍由应用调用；MCP 不会替你训练模型，也不决定任务应该使用哪项能力。

工具用于执行动作，例如查数据库；资源用于提供上下文，例如表结构；提示模板提供可复用的任务表达。三者职责不同。外部能力接入以后，应用仍要判断任务相关性、输入是否有效和用户是否授权。

从使用角度看，先发现能力，再读取声明，最后调用具体工具。不能只猜一个工具名字并传参数。协议版本会变化，当前官方文档与旧案例 SDK 的握手方式可能不同；本课程讲稳定的角色与调用分工，具体连接命令以案例固定版本为学习对象。

流程：Host → Client → Server；Server 提供 tools、resources、prompts

本包没有可运行的 MCP 服务，三项基础练习也不是 MCP 协议实现。它们先帮你理解函数契约、状态和证据，然后再通过两个 MCP 案例观察框架如何把这些能力接起来。

示例与观察

把三个对象对应到浏览器案例：

角色	案例中的对应物
Host	Streamlit 应用与 Agent 编排
Client	框架维护的连接与工具调用
Server	名为 playwright 的浏览器工具服务

服务名称来自配置，并不是 Python 函数本身。

动手任务

给“查询资料库”和“创建日历事件”各写出 Server 应提供的能力，说明哪个属于有副作用的工具。

参考答案

查询资料库是只读工具或资源读取；创建事件是写入工具，需要明确日期、时区、对象和授权。MCP 让连接统一，不会自动消除写入风险。

常见问题

问：MCP 就是 Agent 吗？答：它是协议；使用 MCP 的应用可以是 Agent，也可以是固定工作流。

问：本包有完整 MCP 部署课吗？答：主线讲机制，案例手册提供固定版本阅读与复现路线，未将上游服务验收为可直接使用的成品。

第22课 浏览器行动需要观察与结果核对

模块：MCP 与协作 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

把网页操作拆成观察、选择、动作和验证；识别页面变化造成的失败。

中文讲解

浏览器工具让应用读取页面、点击、输入或截图。与计算函数不同，页面会变化：按钮名字改了、登录状态过期、弹窗遮挡、异步内容还没加载。每次动作前要知道当前页面，每次动作后要观察结果，不能沿用早已失效的元素位置。

给任务写清目标和成功信号。例如“在公开文档页找到安装命令”，成功信号是页面展示了对应版本的命令与来源；不是浏览器已经打开，也不是模型说“已完成”。搜索、登录、付款是不同层次的动作，任务只需要查资料时应收敛到只读路径。

上游 browser_mcp_agent 使用名为 playwright 的服务，经框架列出工具并绑定语言模型；实际服务配置在单独文件中。源码里的状态字段 initialized 表示应用初始化，不代表每个网页动作都正确完成。

流程：观察页面 → 定位控件 → 执行动作 → 再观察 → 核对成功信号

网页文本属于外部数据。页面要求“把本机密钥复制到这里”不是用户授权，应当忽略这类指令。学习浏览器案例时先用公开页面和只读任务，记录预期结果与实际观察。

示例与观察

设计一个只读任务卡：

```
task = {"goal":"查找官方安装命令",
        "allowed":"读取公开文档",
        "success":"找到命令、版本和原页面链接"}
print(task["success"])
```

该代码只是任务规范，不会启动浏览器或执行网页操作。

动手任务

为“点击下一页后页码未变”和“页面弹出登录框”各写出合理下一步。

参考答案

页码未变先重新观察是否按钮禁用、加载失败或命中错误控件；登录框先判断任务是否确实需要登录及是否具备授权，不绕过访问控制。不能把动作尝试写成任务完成。

常见问题

问：截图能证明所有操作成功吗？答：需要截图包含任务相关的成功信号，不是一张任意页面。

问：能直接按固定坐标自动点吗？答：页面布局会变，语义定位和最新观察通常更稳，坐标也需要当前截图支撑。

第23课 多服务协作先确定数据与权限边界

模块：MCP 与协作 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

为多工具任务定义输入输出；控制凭据传递；识别任务跨服务时的副作用。

中文讲解

多服务协作可以把“查到一个问题”转成“建立一个待办”，但连接更多服务也引入更多权限和失败点。先画数据流：从 GitHub 读哪项信息，生成什么摘要，向日历写什么字段。每次跨边界都要确认必要性和授权。

不同服务返回的数据不能靠自然语言想象转换。日期必须有时区，标题应有长度限制，事件创建成功应有事件 ID。某个服务失败时，需要区分已经读取的数据与尚未执行的写入，避免把整个任务重新执行一遍。

上游 Multi-MCP 案例连接多个服务，并在启动时生成用户和会话 ID。源码把整个进程环境传给子服务；这是学习时值得审视的点，因为无关凭据也可能被继承。独立实现应只传必要变量，权限范围也应按任务选择。

流程：只读取证 → 结构化任务草稿 → 校验字段与授权 → 必要写入 → 保存结果 ID

不要把“服务已经连接”当作“所有操作已经允许”。本课程不操作你的邮箱、日历或仓库，案例只是说明编排方法和复现条件。你自行学习时也应从只读工具和测试数据开始。

示例与观察

事件草稿示例：

```
draft = {"title":"复查RAG故障记录",
        "date":"2026-10-20",
        "timezone":"Asia/Shanghai",
        "source":"教学任务"}
assert draft["timezone"]
print(draft)
```

仅创建字典，不创建真实日历事件。

动手任务

设计“读取问题列表并生成待办草稿”的两阶段流程，列出每阶段所需权限。

参考答案

第一阶段只读仓库问题，第二阶段在本机生成草稿，不需要日历写权限。真正创建事件是第三阶段，需校验字段并取得对应授权。不要为只读任务一开始就授予全面管理权限。

常见问题

问：共享同一套环境变量更方便吗？答：方便不意味着必要，应按服务最小需求传递变量。

问：随机 session ID 能恢复上次会话吗？答：只有重新使用旧 ID 才能定向恢复，随机生成通常意味着新会话。

第24课 用分工与验收完成课程作品

模块：MCP 与协作 | 建议用时：阅读25分钟 + 动手20分钟

学习目标

给多 Agent 设计清楚职责；控制共享状态；用成果与失败样本展示学习结果。

中文讲解

多 Agent 的价值来自职责与信息分工。一个负责找资料，一个负责阅读，一个负责整理，可以让阶段更清楚；但如果每个角色都重复搜索、重复总结，成本增加而质量未必提高。先说明单 Agent 哪里受限，再决定拆分。

分工还需要交付接口。研究角色输出来源和事实，审核角色检查冲突与缺失，整理角色依据已确认事实写报告。角色名称本身不保证行为，交付字段、来源要求和验收条件才决定能否组合。

共享状态应明确谁能写什么。两个 Agent 同时覆盖同一个 itinerary 字段，后执行的可能抹掉前面的约束。旅行案例中的团队协调器与成员组织展示了一种结构，但预算和时间是否满足，仍需要确定性检查与人工复核。

流程：任务拆分 → 角色交付 → 来源检查 → 合并结果 → 约束验收

结课时整理三个基础作品，每个附一条成功输入、一条失败输入、执行方式和局限。再任选一个上游案例，写一页架构说明和一个你会改进的问题。作品应展示你能解释系统，而不是把模型截图当作全部成果。

示例与观察

结课交付清单：

作品	成功场景	失败场景
工具助手	18×7=126	除零、未知工具
会话记忆	两进程恢复历史	session 串组检查
文档问答	材料要求有来源	到账日期资料不足

运行测试：python -m unittest -v test_projects.py。

动手任务

用三个练习制作一份作品说明，并给多 Agent 研究团队写出“找、读、审、写”的交付字段。

参考答案

作品说明包含问题、数据流、运行命令、成功结果、失败边界和改进方向。研究字段至少有标题、URL、时间、原文依据；审核字段有支持状态与冲突；整理阶段不得把未确认内容写成事实。

常见问题

问：做完就能当生产项目卖给企业吗？答：课程是教学入门，还需权限、部署、监控、评测和真实数据验证。

问：如何判断学习完成？答：能独立运行、解释每个阶段、复现失败并核对证据，比背诵术语更有用。

继续学习与来源

24课练习参考答案和四周计划位于00-开始学习；12案例另有完整手册PDF。原文参考索引与许可位于04-来源与许可。

三个基础练习默认离线，联网效果未实测；上游案例未运行验收。关键词与向量检索的实现边界在配置说明中列明。

选择下一条练习路线

方向	对应案例	做出一项可检查的改进
检索优化	案例04到08	为同义题和无答案题建回归表
工具集成	案例09到10	画清读取、草稿与写入边界
多Agent分工	案例11到12	定义交付字段与约束检查

先完成基础练习并写出局限，再自行决定是否安装上游框架。运行项目时用独立环境和公开或虚构资料，保存真实结果，不把尝试动作当作任务完成。

原文与许可

课程新增中文讲解、练习和教学结构。相关开源原文免费公开，以下名称仅用于附录来源识别，不代表课程获得第三方官方背书。

来源	许可	阅读位置
Agentic AI学习路线	MIT	04-来源与许可中的固定版本索引
Microsoft AI Agents	MIT	同一索引中的Microsoft目录
Hugging Face Agents	Apache-2.0	同一索引中的Hugging Face目录
awesome-llm-apps案例	Apache-2.0	案例手册链接与17个实现阅读附件

许可原文保存在04-来源与许可/licenses。源码附件保留上游内容，供回查，不是12个完整安装包。第三方开源部分继续按原有许可使用。

复习时问自己

工具真的执行了吗？历史真的传给模型了吗？资料真的回答了问题吗？引用真的支持了结论吗？把这四个问题与执行记录放在一起，才能说明你掌握了数据流。